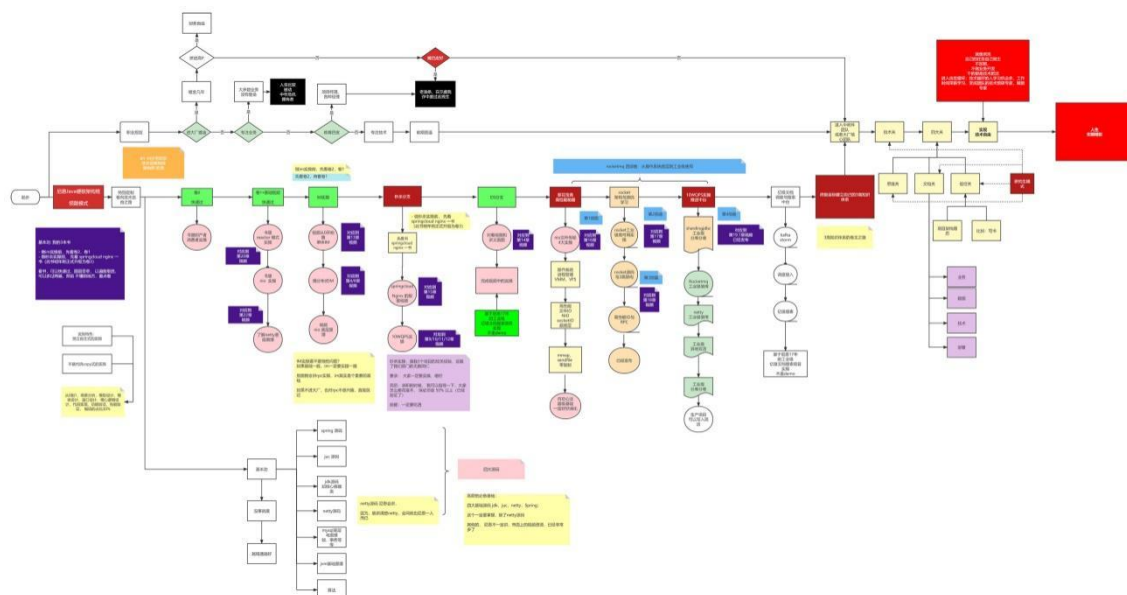


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

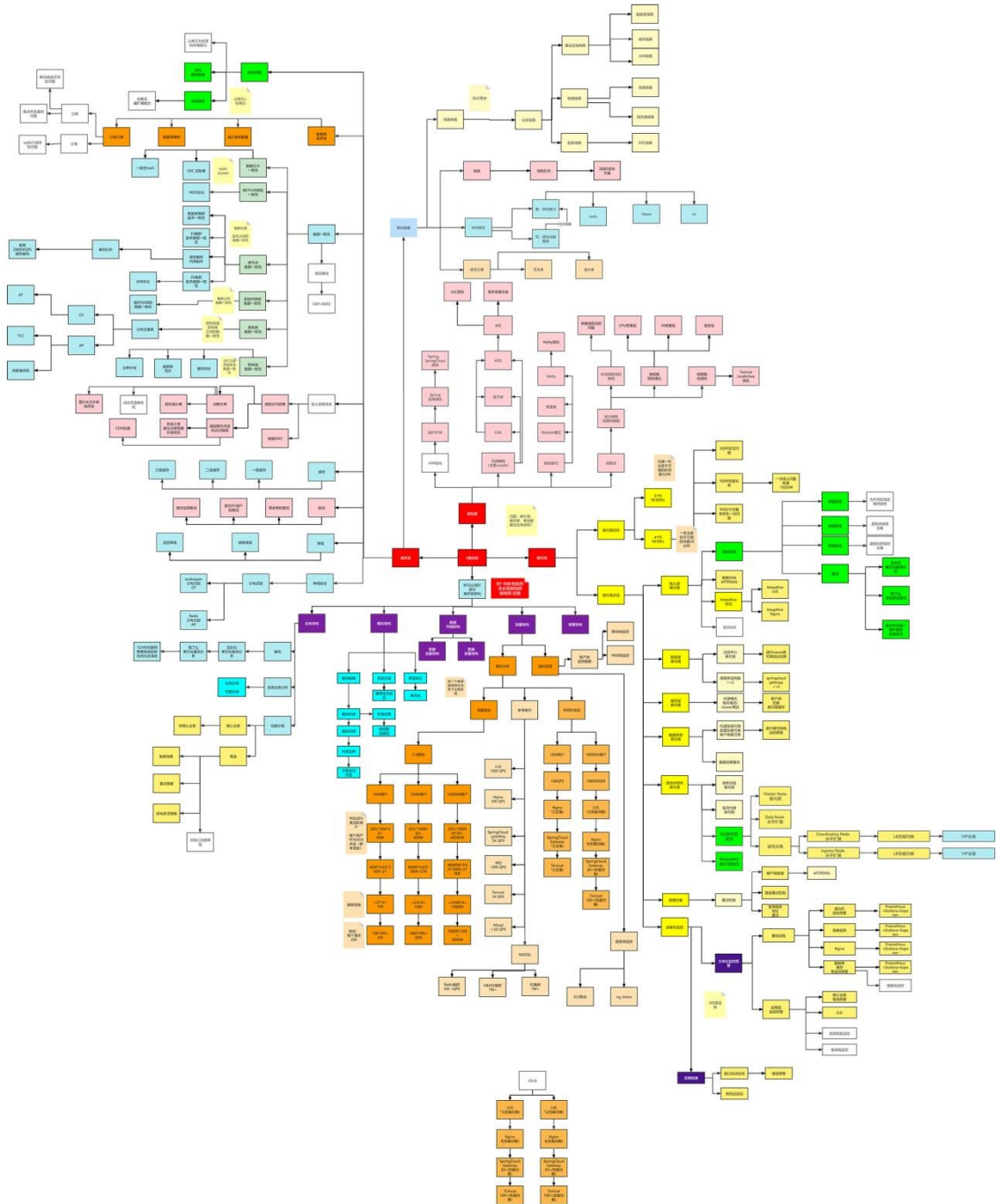
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

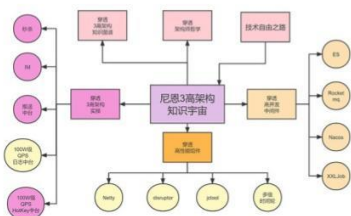
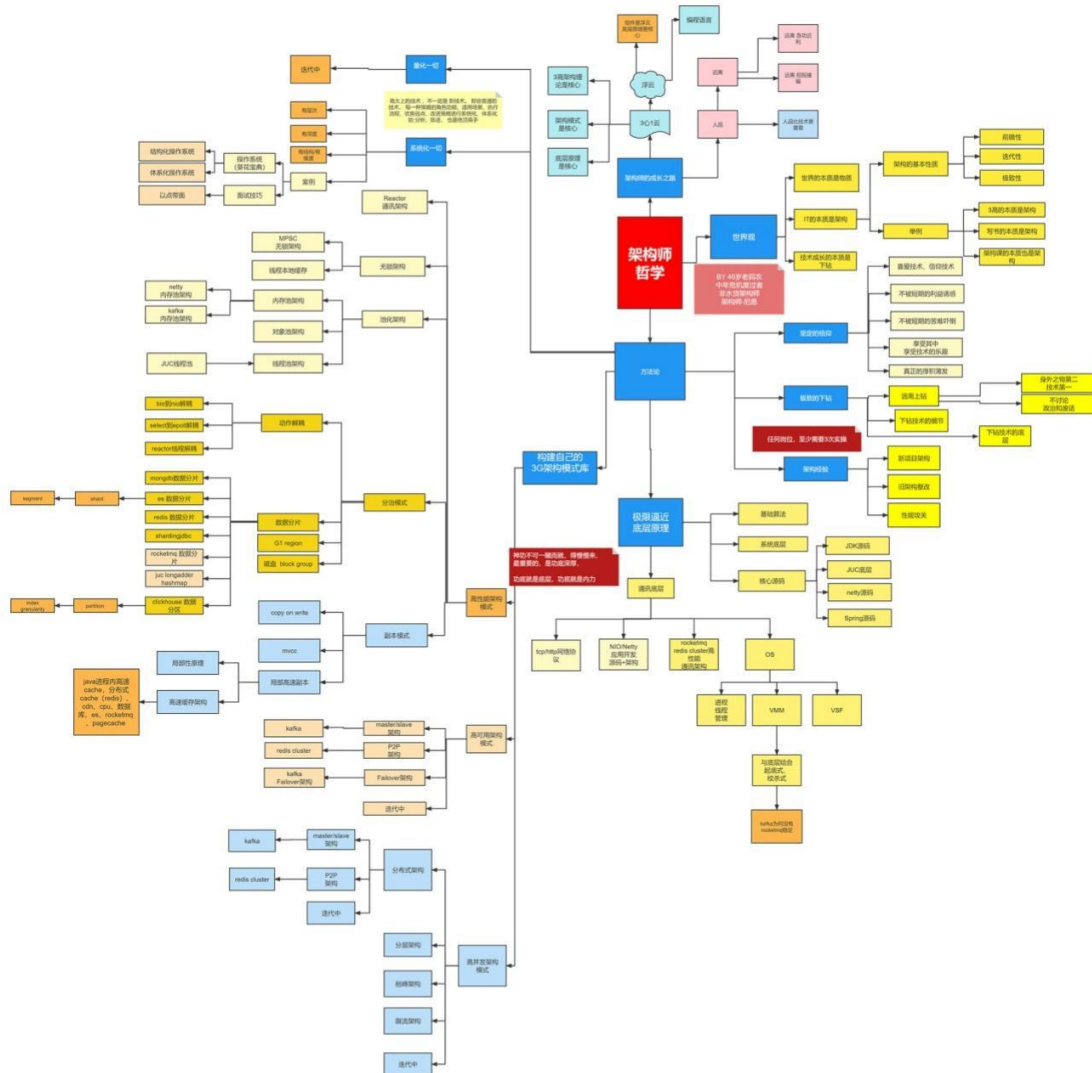
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



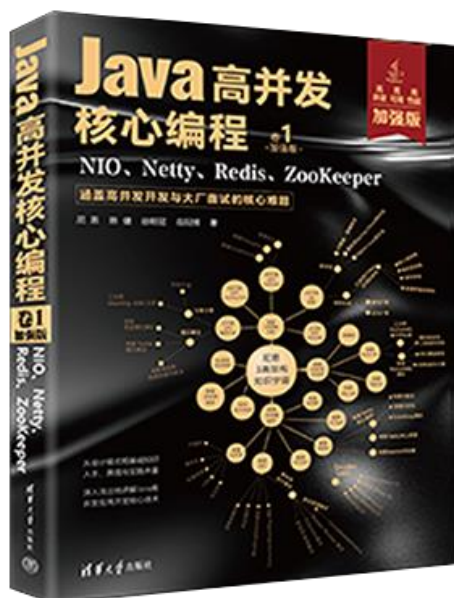
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



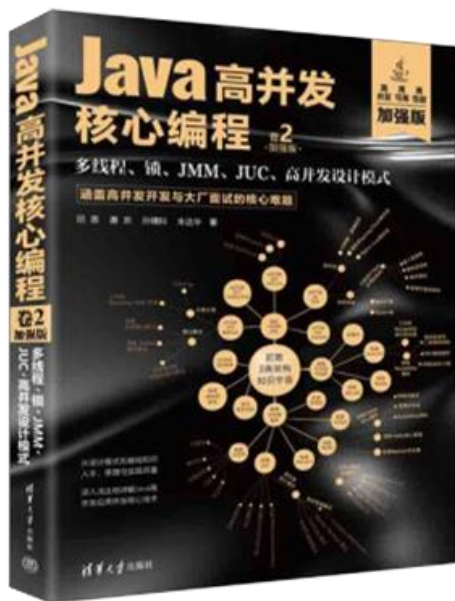
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂地介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



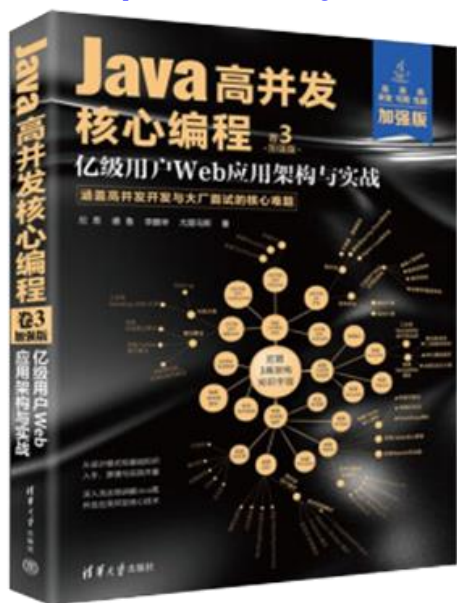
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 Spring Cloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



专题13：死锁面试题（史上最全、定期更新）

本文版本说明：V2

此文的格式，由markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《Java面试红宝书》，后面会不断升级，迭代。

本专题，作为 《Java面试红宝书》专题之一，《Java面试红宝书》一共**30个面试专题**，后续还会增加

《Java面试红宝书》升级的规划为：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫架构师尼恩微信，发送“领取电子书”获取。

尼恩的微信二维码在哪里呢？请参见文末

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈社群交流，加入交流群，加尼恩微信即可，

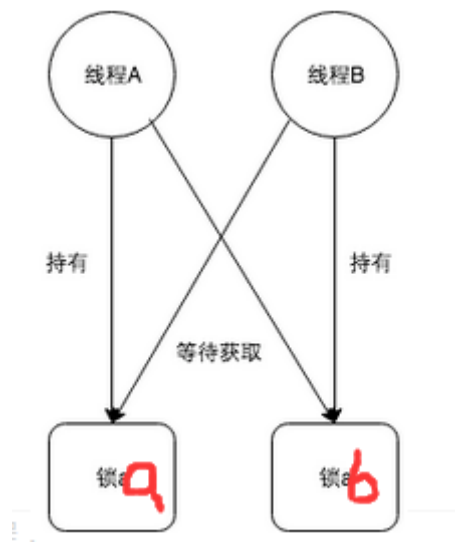
前言

首先介绍大厂的死锁面试题，然后对死锁做一个全面的解读。

大厂的死锁面试题

什么是死锁？

所谓死锁，是指多个进程在运行过程中因争夺资源而造成的一种僵局，当进程处于这种僵持状态时，若无外力作用，它们都将无法再向前推进。因此我们举个例子来描述，如果此时有一个线程A，按照先锁a再获得锁b的顺序获得锁，而在此同时又有另外一个线程B，按照先锁b再锁a的顺序获得锁。如下图所示：



产生死锁的原因？

可归结为如下两点：

a. 竞争资源

- 系统中的资源可以分为两类：
 1. 可剥夺资源，是指某进程在获得这类资源后，该资源可以再被其他进程或系统剥夺，CPU和主存均属于可剥夺性资源；
 2. 另一类资源是不可剥夺资源，当系统把这类资源分配给某进程后，再不能强行收回，只能在进程用完后自行释放，如磁带机、打印机等。
- 产生死锁中的竞争资源之一指的是竞争不可剥夺资源（例如：系统中只有一台打印机，可供进程P1使用，假定P1已占用了打印机，若P2继续要求打印机打印将阻塞）
- 产生死锁中的竞争资源另外一种资源指的是竞争临时资源（临时资源包括硬件中断、信号、消息、缓冲区内的消息等），通常消息通信顺序进行不当，则会产生死锁

b. 进程间推进顺序非法

- 若P1保持了资源R1,P2保持了资源R2，系统处于不安全状态，因为这两个进程再向前推进，便可能发生死锁
- 例如，当P1运行到P1: Request (R2) 时，将因R2已被P2占用而阻塞；当P2运行到P2: Request (R1) 时，也将因R1已被P1占用而阻塞，于是发生进程死锁

死锁产生的4个必要条件？

产生死锁的必要条件：

1. 互斥条件：进程要求对所分配的资源进行排它性控制，即在一段时间内某资源仅为一进程所占用。
2. 请求和保持条件：当进程因请求资源而阻塞时，对已获得的资源保持不放。
3. 不剥夺条件：进程已获得的资源在未使用完之前，不能剥夺，只能在使用完时由自己释放。
4. 环路等待条件：在发生死锁时，必然存在一个进程--资源的环形链。

解决死锁的基本方法

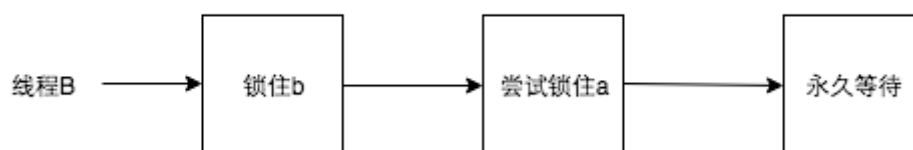
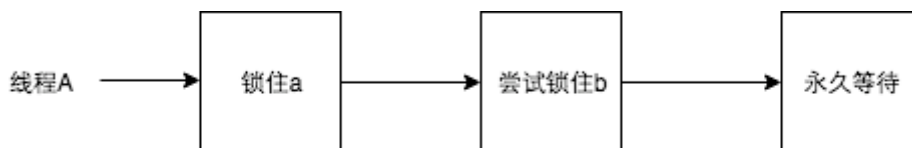
一、预防死锁：

- 资源一次性分配：一次性分配所有资源，这样就不会再有请求了：（破坏请求条件）
- 只要有一个资源得不到分配，也不给这个进程分配其他的资源：（破坏请保持条件）

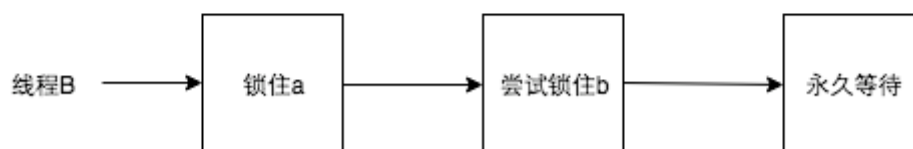
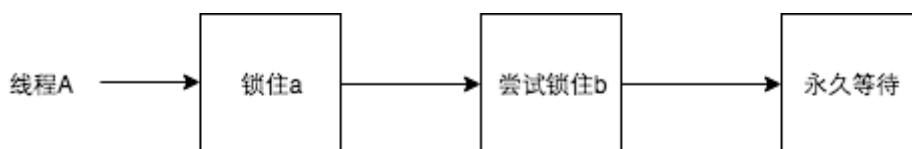
- 可剥夺资源：即当某进程获得了部分资源，但得不到其它资源，则释放已占有的资源（破坏不可剥夺条件）
- 资源有序分配法：系统给每类资源赋予一个编号，每一个进程按编号递增的顺序请求资源，释放则相反（破坏环路等待条件）

1 以确定的顺序获得锁

如果必须获取多个锁，那么在设计的时候需要充分考虑不同线程之前获得锁的顺序。按照上面的例子，两个线程获得锁的时序图如下：



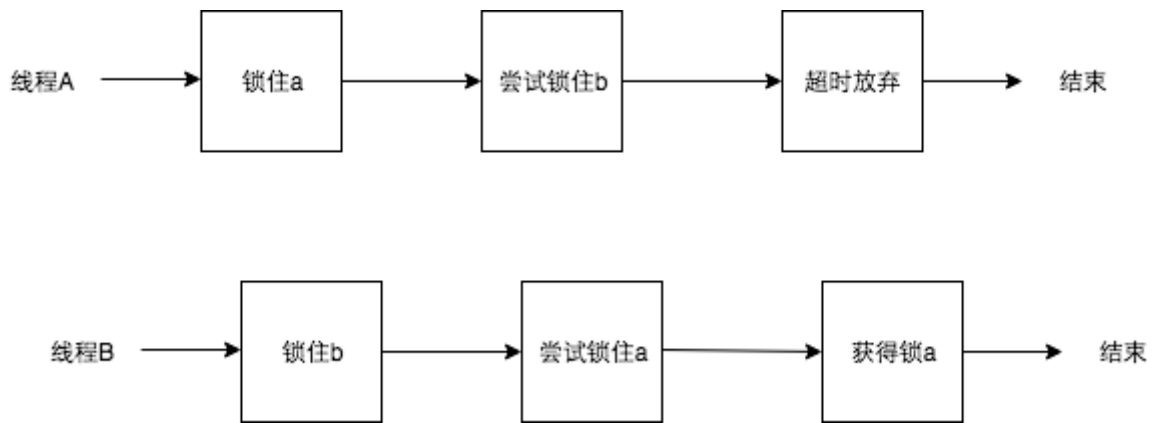
如果此时把获得锁的时序改成：



那么死锁就永远不会发生。针对两个特定的锁，开发者可以尝试按照锁对象的hashCode值大小的顺序，分别获得两个锁，这样锁总是会以特定的顺序获得锁，那么死锁也不会发生。问题变得更加复杂一些，如果此时有多个线程，都在竞争不同的锁，简单按照锁对象的hashCode进行排序（单纯按照hashCode顺序排序会出现“环路等待”），可能就无法满足要求了，这个时候开发者可以使用银行家算法，所有的锁都按照特定的顺序获取，同样可以防止死锁的发生，该算法在这里就不再赘述了，有兴趣的可以自行了解一下。

2 超时放弃

当使用synchronized关键词提供的内置锁时，只要线程没有获得锁，那么就会永远等待下去，然而Lock接口提供了boolean tryLock(long time, TimeUnit unit) throws InterruptedException方法，该方法可以按照固定时长等待锁，因此线程可以在获取锁超时以后，主动释放之前已经获得的所有的锁。通过这种方式，也可以很有效地避免死锁。还是按照之前的例子，时序图如下：



二、避免死锁:

- 预防死锁的几种策略，会严重地损害系统性能。因此在避免死锁时，要施加较弱的限制，从而获得较满意的系统性能。由于在避免死锁的策略中，允许进程动态地申请资源。因而，系统在进行资源分配之前预先计算资源分配的安全性。若此次分配不会导致系统进入不安全的状态，则将资源分配给进程；否则，进程等待。其中最具有代表性的避免死锁算法是银行家算法。
- 银行家算法：首先需要定义状态和安全状态的概念。系统的状态是当前给进程分配的资源情况。因此，状态包含两个向量Resource（系统中每种资源的总量）和Available（未分配给进程的每种资源的总量）及两个矩阵Claim（表示进程对资源的需求）和Allocation（表示当前分配给进程的资源）。安全状态是指至少有一个资源分配序列不会导致死锁。当进程请求一组资源时，假设同意该请求，从而改变了系统的状态，然后确定其结果是否还处于安全状态。如果是，同意这个请求；如果不是，阻塞该进程知道同意该请求后系统状态仍然是安全的。

三、检测死锁

1. 首先为每个进程和每个资源指定一个唯一的号码；
2. 然后建立资源分配表和进程等待表。

死锁检测的工具

1、Jstack命令

jstack是java虚拟机自带的一种堆栈跟踪工具。jstack用于打印出给定的java进程ID或core file或远程调试服务的Java堆栈信息。Jstack工具可以用于生成java虚拟机当前时刻的线程快照。线程快照是当前java虚拟机内每一条线程正在执行的方法堆栈的集合，生成线程快照的主要目的是定位线程出现长时间停顿的原因，如线程间死锁、死循环、请求外部资源导致的长时间等待等。线程出现停顿的时候通过jstack来查看各个线程的调用堆栈，就可以知道没有响应的线程到底在后台做什么事情，或者等待什么资源。

2、JConsole工具

Jconsole是JDK自带的监控工具，在JDK/bin目录下可以找到。它用于连接正在运行的本地或者远程的JVM，对运行在Java应用程序的资源消耗和性能进行监控，并画出大量的图表，提供强大的可视化界面。而且本身占用的服务器内存很小，甚至可以说几乎不消耗。

四、解除死锁:

当发现有进程死锁后，便应立即把它从死锁状态中解脱出来，常采用的方法有：

- 剥夺资源：从其它进程剥夺足够数量的资源给死锁进程，以解除死锁状态；
- 撤消进程：可以直接撤消死锁进程或撤消代价最小的进程，直至有足够的资源可用，死锁状态消除为止；所谓代价是指优先级、运行代价、进程的重要性和价值等。

ok, 介绍大厂的死锁面试题之后, 接下来, 对死锁做一个全面的解读。

一、什么是死锁

多线程以及多进程改善了系统资源的利用率并提高了系统的处理能力。然而, 并发执行也带来了新的问题——死锁。

死锁是指两个或两个以上的进程(线程)在运行过程中因争夺资源而造成的一种僵局(Deadly-Embrace), 若无外力作用, 这些进程(线程)都将无法向前推进。

下面我们通过一些实例来说明死锁现象。

先看生活中的一个实例, 2个人一起吃饭但是只有一双筷子, 2人轮流吃(同时拥有2只筷子才能吃)。某一个时候, 一个拿了左筷子, 一人拿了右筷子, 2个人都同时占用一个资源, 等待另一个资源, 这个时候甲在等待乙吃完并释放它占有的筷子, 同理, 乙也在等待甲吃完并释放它占有的筷子, 这样就陷入了一个死循环, 谁也无法继续吃饭。。。

在计算机系统中也存在类似的情况。例如, 某计算机系统中只有一台打印机和一台输入设备, 进程P1正占用输入设备, 同时又提出使用打印机的请求, 但此时打印机正被进程P2所占用, 而P2在未释放打印机之前, 又提出请求使用正被P1占用着的输入设备。这样两个进程相互无休止地等待下去, 均无法继续执行, 此时两个进程陷入死锁状态。

关于死锁的一些结论:

- 参与死锁的进程数至少为两个
- 参与死锁的所有进程均等待资源
- 参与死锁的进程至少有两个已经占有资源
- 死锁进程是系统中当前进程集合的一个子集
- 死锁会浪费大量系统资源, 甚至导致系统崩溃。

举一个例子:

如何解决上面的问题呢? 正所谓知己知彼方能百战不殆, 我们要先了解什么情况会发生死锁, 才能知道如何避免死锁, 很幸运我们可以站在巨人的肩膀上看待问题

一个银行转账经典案例:

账户A给账户B转账, 账户A余额减少100元, 账户B余额增加100元, 这个操作要是原子性的

先来看程序:

```
class Account {
    private int balance;
    // 转账
    synchronized void transfer(
        Account target, int amt){
        if (this.balance > amt) {
            this.balance -= amt;
            target.balance += amt;
        }
    }
}
```

用 synchronized 直接保护 transfer 方法，然后操作 资源「Account 的A 余额」和资源「Account的B 余额」就可以了

其中有两个问题:

1. 单纯的用 synchronized 方法起不到保护作用(不能保护 target)
2. 用 Account.class 锁方案，锁的粒度又过大，导致涉及到账户的所有操作(取款，转账，修改密码等)都会变成串行操作

如何解决这两个问题呢？咱们先换好衣服穿越回到过去寻找一下钱庄，一起透过现象看本质，dengdeng deng.....



来到钱庄，告诉柜员你要给铁蛋儿转 100 铜钱，这时柜员转身在墙上寻找你和铁蛋儿的账本，此时柜员可能面临三种情况:

1. 理想状态: 你和铁蛋儿的账本都是空闲状态，一起拿回来，在你的账本上减 100 铜钱，在铁蛋儿账本上加 100 铜钱，柜员转身将账本挂回到墙上，完成你的业务

2. 尴尬状态: 你的账本在, 铁蛋儿的账本被其他柜员拿出去给别人转账, 你要等待其他柜员把铁蛋儿的账本归还

3. 抓狂状态: 你的账本不在, 铁蛋儿的账本也不在, 你只能等待两个账本都归还

放慢柜员的取账本操作, 他一定是先拿到你的账本, 然后再去拿铁蛋儿的账本, 两个账本都拿到(理想状态)之后才能完成转账, 用程序模型来描述一下这个拿取账本的过程:



我们继续用程序代码描述一下上面这个模型:

```
class Account {  
    private int balance;  
    // 转账  
    void transfer(Account target, int amt){  
        // 锁定转出账户  
        synchronized(this) {  
            // 锁定转入账户
```

```

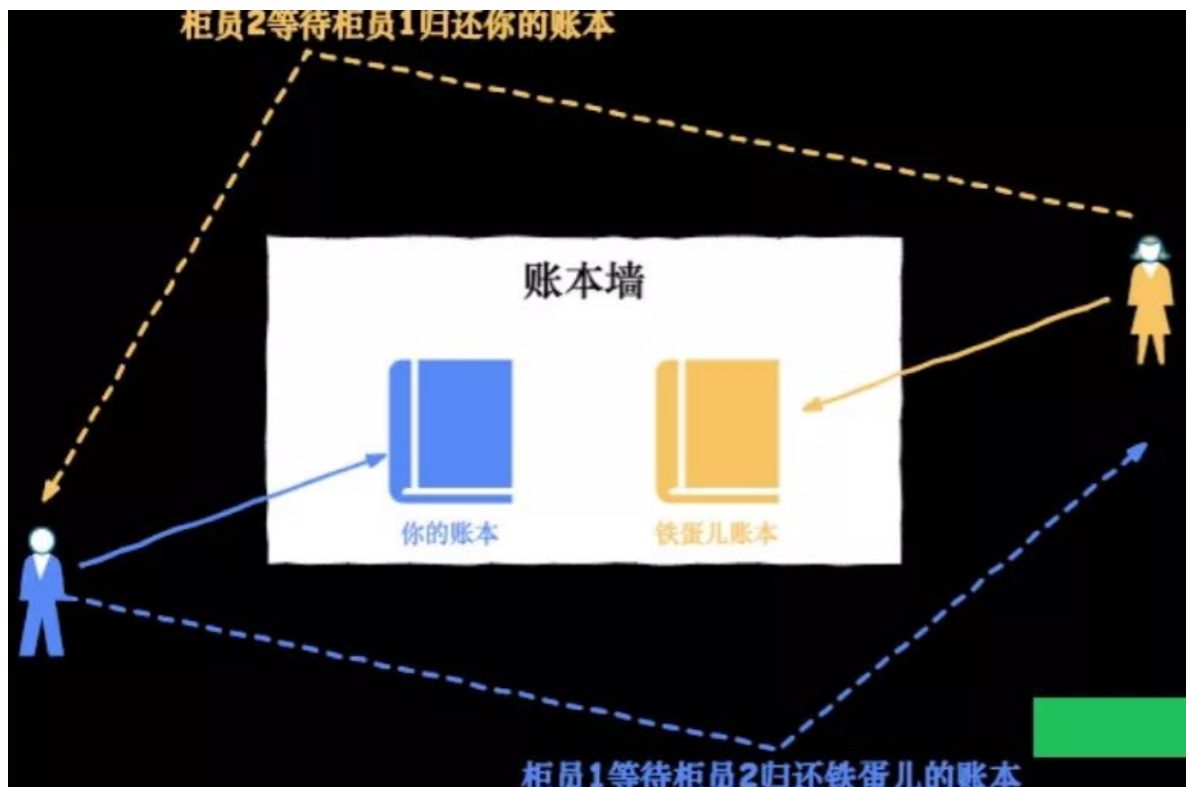
synchronized(target) {
    if (this.balance > amt) {
        this.balance -= amt;
        target.balance += amt;
    }
}
}
}
}

```

这个解决方案看起来很完美，解决了文章开头说的两个问题，但真是这样吗？

我们刚刚说过的理想状态是钱庄只有一个柜员(既单线程)。随着钱庄规模变大，墙上早已挂了非常多个账本，钱庄为了应对繁忙的业务，开通了多个窗口，此时有多个柜员(多线程)处理钱庄业务。

柜员 1 正在办理给铁蛋儿转账的业务，但只拿到了你的账本；柜员 2 正在办理铁蛋儿给你转账的业务，但只拿到了铁蛋儿的账本，此时双方出现了尴尬状态，两位柜员都在等待对方归还账本为当前客户办理转账业务。



现实中柜员会沟通，喊出一嗓子 **老铁，铁蛋儿的账本先给我用一下，用完还给你**，但程序却没这么智能，synchronized 内置锁非常执着，它会告诉你「死等」的道理，最终出现死锁

如何解决死锁呢？

Java 有了 synchronized 内置锁，还发明了显示锁 Lock，是不是就为了治一治 synchronized 「死等」的执着呢？

如果你捉急，可以直接去阅读第四节。

二、死锁与饥饿

饥饿 (Starvation) 指一个进程一直得不到资源。

死锁和饥饿都是由于进程竞争资源而引起的。饥饿一般不占有资源，死锁进程一定占有资源。

三、资源的类型

3.1 可重用资源和消耗性资源

3.1.1 可重用资源（永久性资源）

可被多个进程多次使用，如所有硬件。

- 只能分配给一个进程使用，不允许多个进程共享。
- 进程在对可重用资源的使用时，须按照请求资源、使用资源、释放资源这样的顺序。
- 系统中每一类可重用资源中的单元数目是相对固定的，进程在运行期间，既不能创建，也不能删除。

3.1.2 消耗性资源（临时性资源）

又称临时性资源，是由进程在运行期间动态的创建和消耗的。

- 消耗性资源在进程运行期间是可以不断变化的，有时可能为0。
- 进程在运行过程中，可以不断地创造可消耗性资源的单元，将它们放入该资源类的缓冲区中，以增加该资源类的单元数目。
- 进程在运行过程中，可以请求若干个可消耗性资源单元，用于进程自己消耗，不再将它们返回给该资源类中。

可消耗资源通常是由生产者进程创建，由消费者进程消耗。最典型的可消耗资源是用于进程间通信的消息。

3.2 可抢占资源和不可抢占资源

3.2.1 可抢占资源

可抢占资源指某进程在获得这类资源后，该资源可以再被其他进程或系统抢占。对于这类资源是不会引起死锁的。

CPU 和主存均属于可抢占性资源。

3.2.2 不可抢占资源

一旦系统把某资源分配给该进程后，就不能将它强行收回，只能在进程用完后自行释放。

磁带机、打印机等属于不可抢占性资源。

四、死锁产生的原因

- **竞争不可抢占资源引起死锁**

通常系统中拥有的不可抢占资源，其数量不足以满足多个进程运行的需要，使得进程在运行过程中，会因争夺资源而陷入僵局，如磁带机、打印机等。只有对不可抢占资源的竞争 才可能产生死锁，对可抢占资源的竞争是不会引起死锁的。

- **竞争可消耗资源引起死锁**

- **进程推进顺序不当引起死锁**

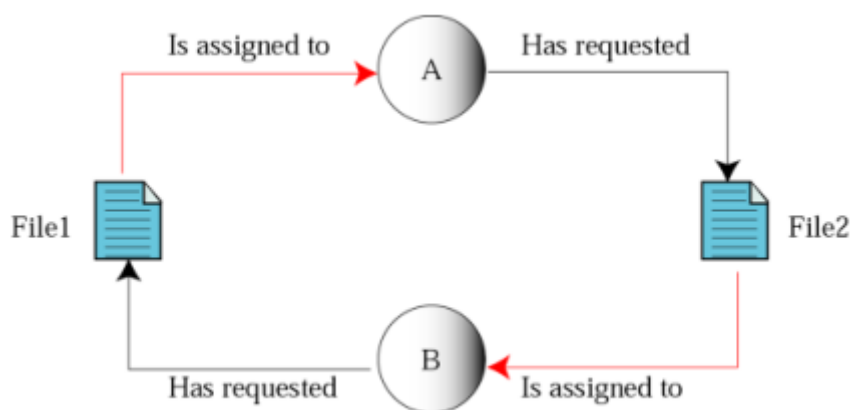
进程在运行过程中，请求和释放资源的顺序不当，也同样会导致死锁。例如，并发进程 P1、P2 分别保持了资源 R1、R2，而进程 P1 申请资源 R2，进程 P2 申请资源 R1 时，两者都会因为所需资源被占用而阻塞。

信号量使用不当也会造成死锁。进程间彼此相互等待对方发来的消息，结果也会使得这些进程间无法继续向前推进。例如，进程 A 等待进程 B 发的消息，进程 B 又在等待进程 A 发的消息，**可以看出进程 A 和 B 不是因为竞争同一资源，而是在等待对方的资源导致死锁。**

4.1 竞争不可抢占资源引起死锁

如：共享文件时引起死锁

系统中拥有两个进程 P1 和 P2，它们都准备写两个文件 F1 和 F2。而这两者都属于可重用和不可抢占性资源。如果进程 P1 在打开 F1 的同时，P2 进程打开 F2 文件，当 P1 想打开 F2 时由于 F2 已被占用而阻塞，当 P2 想打开 F1 时由于 F1 已被占用而阻塞，此时就会无限等待下去，形成死锁。

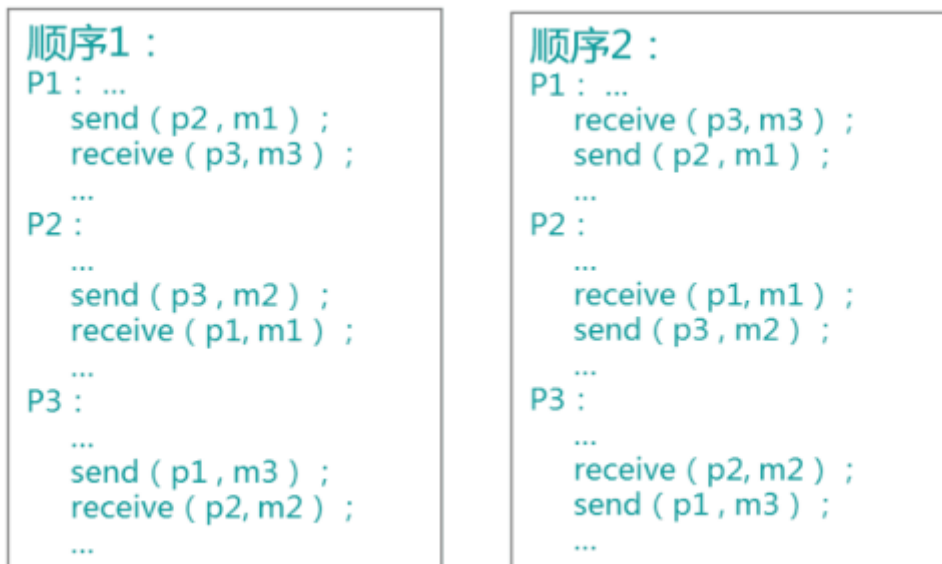


4.2 竞争可消耗资源引起死锁

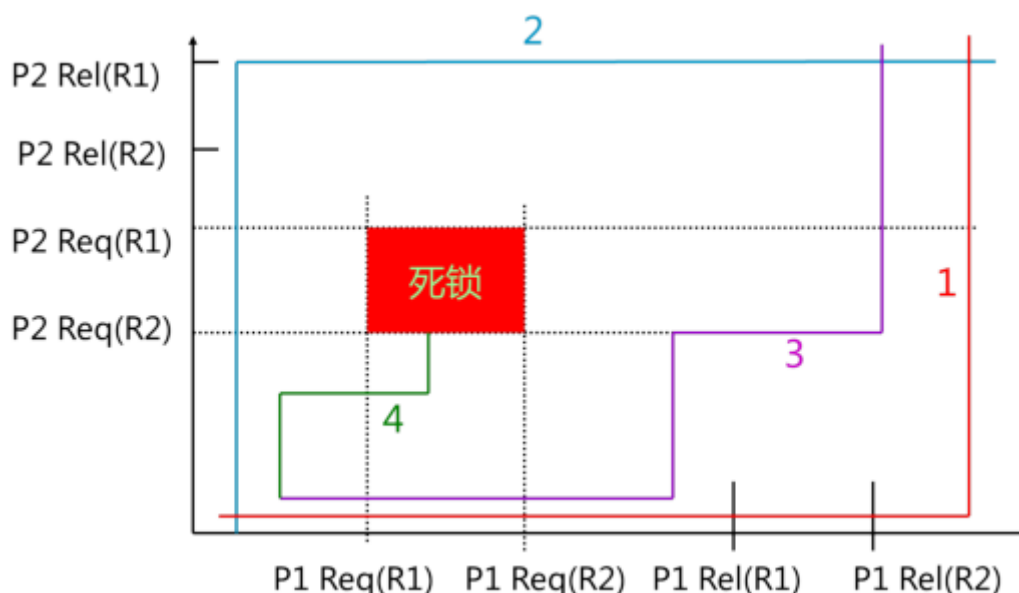
如：进程通信时引起死锁

系统中拥有三个进程 P1、P2 和 P3，m1、m2、m3 是 3 可消耗资源。进程 P1 一方面产生消息 m1，将其发送给 P2，另一方面要从 P3 接收消息 m3。而进程 P2 一方面产生消息 m2，将其发送给 P3，另一方面要从 P1 接收消息 m1。类似的，进程 P3 一方面产生消息 m3，将其发送给 P1，另一方面要从 P2 接收消息 m2。如果三个进程都先发送自己产生的消息后接收别人发来的消息，则可以顺利的运行下去不会产生死锁，

但要是三个进程都先接收别人的消息而不产生消息则会永远等待下去，产生死锁。



4.3 进程推进顺序不当引起死锁



上图中，如果按曲线1的顺序推进，两个进程可顺利完成；如果按曲线2的顺序推进，两个进程可顺利完成；如果按曲线3的顺序推进，两个进程可顺利完成；如果按曲线4的顺序推进，两个进程将进入不安全区D中，此时P1保持了资源R1，P2保持了资源R2，系统处于不安全状态，如果继续向前推进，则可能产生死锁。

五、产生死锁的四个必要条件

Coffman 总结出了四个条件说明可以发生死锁的情形：

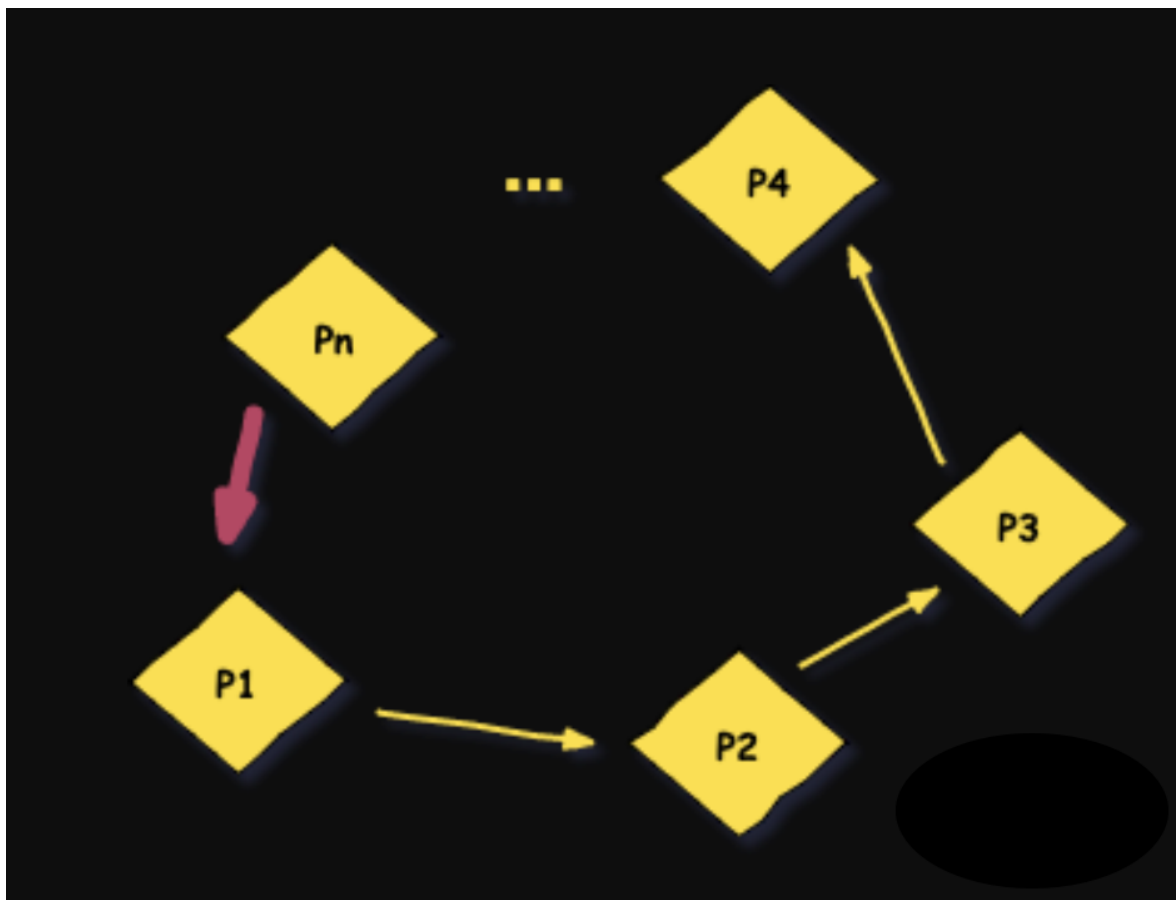
Coffman 条件

互斥条件：指进程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。如果此时还有其它进程请求资源，则请求者只能等待，直至占有资源的进程用毕释放。

不可剥夺条件：指进程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。

请求和保持条件：指进程已经保持至少一个资源，但又提出了新的资源请求，而该资源已被其它进程占有，此时请求进程阻塞，但又对自己已获得的其它资源保持不放。

环路等待条件：指在发生死锁时，必然存在一个进程——资源的环形链，即进程集合{P1, P2, ..., Pn}中的 P1 正在等待一个 P2 占用的资源；P2 正在等待 P3 占用的资源，....., Pn 正在等待已被 P0 占用的资源。



这几个条件很好理解，其中「互斥条件」是并发编程的根基，这个条件没办法改变。但其他三个条件都有改变的可能，也就是说破坏另外三个条件就不会出现上面说到的死锁问题

5.1 互斥条件：

进程要求对所分配的资源（如打印机）进行排他性控制，即在一段时间内某资源仅为一个进程所占有。此时若有其他进程请求该资源，则请求进程只能等待。

5.2 不可剥夺条件：

进程所获得的资源在未使用完毕之前，不能被其他进程强行夺走，即只能由获得该资源的进程自己来释放（只能是主动释放）。

5.3 请求与保持条件：

进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占有，此时请求进程被阻塞，但对自己已获得的资源保持不放。

5.4 循环等待条件：

存在一种进程资源的循环等待链，链中每一个进程已获得的资源同时被链中下一个进程所请求。即存在一个处于等待状态的进程集合 $\{P_1, P_2, \dots, P_n\}$ ，其中 P_i 等待的资源被 P_{i+1} 占有 ($i=0, 1, \dots, n-1$)， P_n 等待的资源被 P_0 占有，如图2-15所示。

直观上看，循环等待条件似乎和死锁的定义一样，其实不然。按死锁定义构成等待环所要求的条件更严，它要求 P_i 等待的资源必须由 P_{i+1} 来满足，而循环等待条件则无此限制。例如，系统中有两台输出设备， P_0 占有一台， P_K 占有另一台，且 K 不属于集合 $\{0, 1, \dots, n\}$ 。

P_n 等待一台输出设备，它可以从 P_0 获得，也可能从 P_K 获得。因此，虽然 P_n 、 P_0 和其他一些进程形成了循环等待圈，但 P_K 不在圈内，若 P_K 释放了输出设备，则可打破循环等待，如图2-16所示。因此循环等待只是死锁的必要条件。

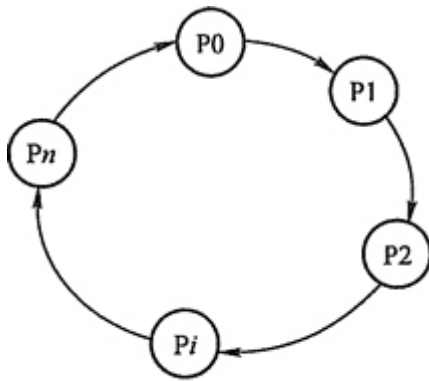


图2-15 循环等待

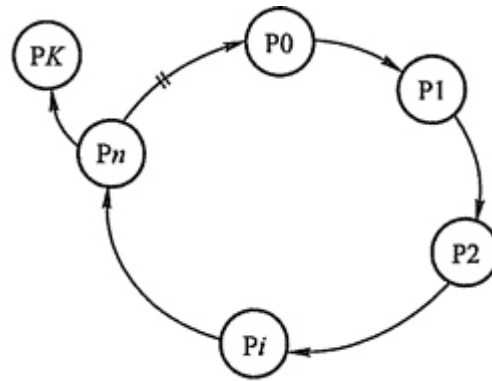


图2-16 满足条件但无死循环

资源分配图含圈而系统又不一定有死锁的原因是同类资源数大于1。但若系统中每类资源都只有一个资源，则资源分配图含圈就变成了系统出现死锁的充分必要条件。

以上这四个条件是死锁的必要条件，只要系统发生死锁，这些条件必然成立，而只要上述条件之一不满足，就不会发生死锁。

产生死锁的一个例子:

```
/**
 * 一个简单的死锁类
 * 当DeadLock类的对象flag==1时（td1），先锁定o1,睡眠500毫秒
 * 而td1在睡眠的时候另一个flag==0的对象（td2）线程启动，先锁定o2,睡眠500毫秒
 * td1睡眠结束后需要锁定o2才能继续执行，而此时o2已被td2锁定；
 * td2睡眠结束后需要锁定o1才能继续执行，而此时o1已被td1锁定；
 * td1、td2相互等待，都需要得到对方锁定的资源才能继续执行，从而死锁。
 */
public class DeadLock implements Runnable {
    public int flag = 1;
    //静态对象是类的所有对象共享的
    private static Object o1 = new Object(), o2 = new Object();
    @Override
    public void run() {
        System.out.println("flag=" + flag);
        if (flag == 1) {
            synchronized (o1) {
                try {
                    Thread.sleep(500);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
            synchronized (o2) {
                System.out.println("1");
            }
        }
    }
}
```

```

        }
    }
    if (flag == 0) {
        synchronized (o2) {
            try {
                Thread.sleep(500);
            } catch (Exception e) {
                e.printStackTrace();
            }
            synchronized (o1) {
                System.out.println("0");
            }
        }
    }
}

public static void main(String[] args) {
    DeadLock td1 = new DeadLock();
    DeadLock td2 = new DeadLock();
    td1.flag = 1;
    td2.flag = 0;
    //td1,td2都处于可执行状态，但JVM线程调度先执行哪个线程是不确定的。
    //td2的run()可能在td1的run()之前运行
    new Thread(td1).start();
    new Thread(td2).start();
}
}

```

六、处理死锁的方法

- **预防死锁**：通过设置某些限制条件，去破坏产生死锁的四个必要条件中的一个或几个条件，来防止死锁的发生。
- **避免死锁**：在资源的动态分配过程中，用某种方法去防止系统进入不安全状态，从而避免死锁的发生。
- **检测死锁**：允许系统在运行过程中发生死锁，但可设置检测机构及时检测死锁的发生，并采取适当措施加以清除。
- **解除死锁**：当检测出死锁后，便采取适当措施将进程从死锁状态中解脱出来。

七、预防死锁

通过设置某些限制条件，去破坏产生死锁的四个必要条件中的一个或几个条件，来防止死锁的发生

破坏死锁的条件：

1. 破坏“互斥”条件:

就是在系统里取消互斥。若资源不被一个进程独占使用，那么死锁是肯定不会发生的。但一般来说在所列的四个条件中，“互斥”条件是无法破坏的。因此，在死锁预防里主要是破坏其他几个必要条件，而不去涉及破坏“互斥”条件。

注意：互斥条件不能被破坏，否则会造成结果的不可再现性。

2. 破坏“占有并等待”条件:

破坏“占有并等待”条件，就是在系统中不允许进程在已获得某种资源的情况下，申请其他资源。即要想出一个办法，阻止进程在持有资源的同时申请其他资源。

方法一：创建进程时，要求它申请所需的全部资源，系统或满足其所有要求，或什么也不给它。这是所谓的“一次性分配”方案。

方法二：要求每个进程提出新的资源申请前，释放它所占有的资源。这样，一个进程在需要资源S时，须先把它先前占有的资源R释放掉，然后才能提出对S的申请，即使它可能很快又要用到资源R。

3. 破坏“不可抢占”条件:

破坏“不可抢占”条件就是允许对资源实行抢夺。

方法一：如果占有某些资源的一个进程进行进一步资源请求被拒绝，则该进程必须释放它最初占有的资源，如果有必要，可再次请求这些资源和另外的资源。

方法二：如果一个进程请求当前被另一个进程占有的一个资源，则操作系统可以抢占另一个进程，要求它释放资源。只有在任意两个进程的优先级都不相同的条件下，方法二才能预防死锁。

4. 破坏“循环等待”条件:

破坏“循环等待”条件的一种方法，是将系统中的所有资源统一编号，进程可在任何时刻提出资源申请，但所有申请必须按照资源的编号顺序（升序）提出。这样做就能保证系统不出现死锁。

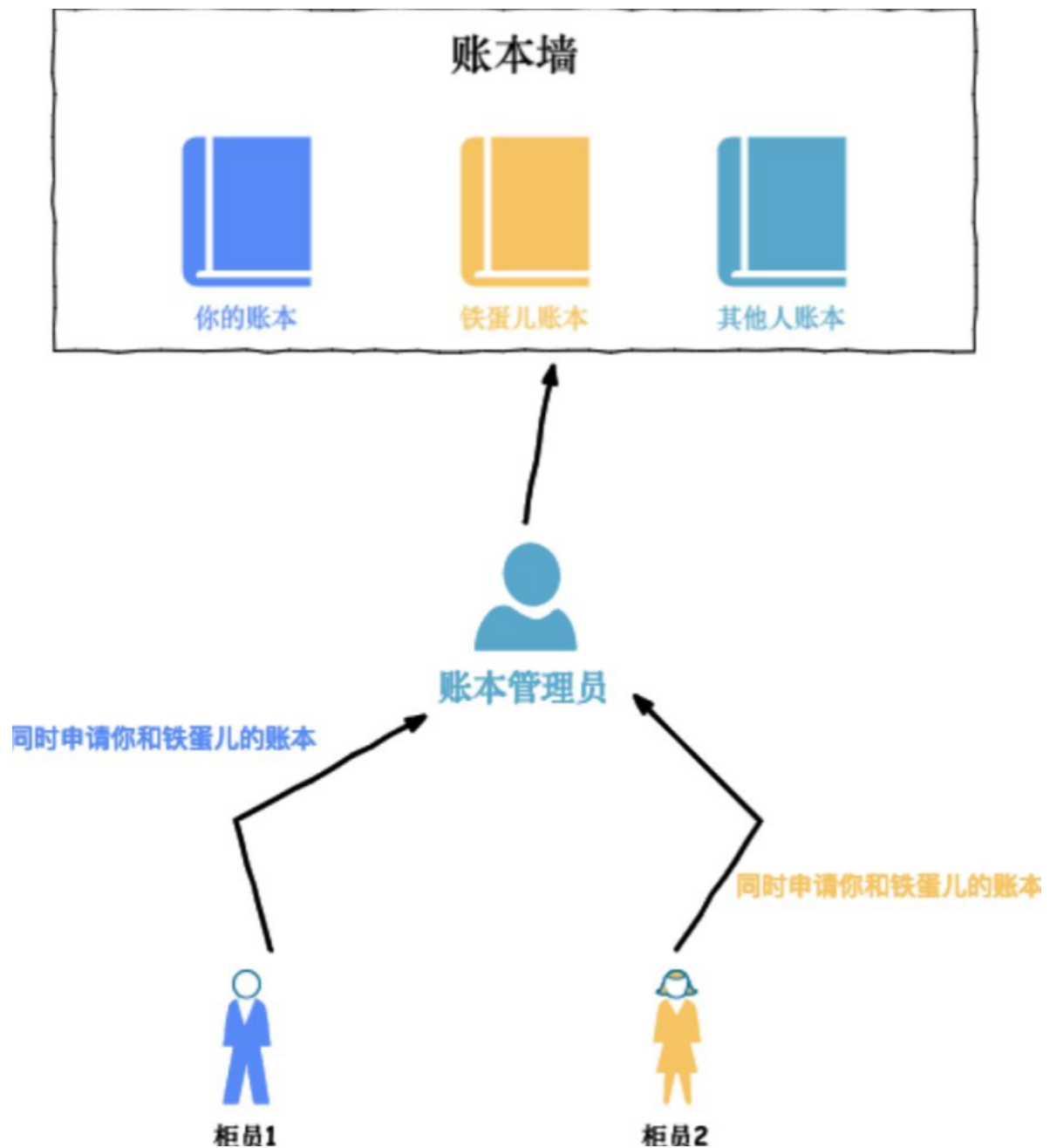
银行转账经典案例中的死锁的避免

解决前面的银行转账经典案例的死锁，有以下方法

方法一：破坏请求和保持条件

每个柜员都可以取放账本，很容易出现互相等待的情况。要想破坏请求和保持条件，就要一次性拿到所有资源。

可以不允许柜员都可以取放账本，账本要由单独的账本管理员来管理



也就是说账本管理员拿取账本是临界区，如果只拿到其中之一的账本，那么不会给柜员，而是等待柜员下一次询问是否两个账本都在

```
//账本管理员
public class AccountBookManager {
    synchronized boolean getAllRequiredAccountBook( Object from, Object to){
        if(拿到所有账本){
            return true;
        } else{
            return false;
        }
    }
    // 归还资源
    synchronized void releaseObtainedAccountBook(Object from, Object to){
        归还获取到的账本
    }
}
```

```

public class Account {
    //单例的账本管理员
    private AccountBookManager accountBookManager;

    public void transfer(Account target, int amt){
        // 一次性申请转出账户和转入账户，直到成功
        while(!accountBookManager.getAllRequiredAccountBook(this, target)){
            return;
        }

        try{
            // 锁定转出账户
            synchronized(this){
                // 锁定转入账户
                synchronized(target){
                    if (this.balance > amt){
                        this.balance -= amt;
                        target.balance += amt;
                    }
                }
            }
        } finally {
            accountBookManager.releaseObtainedAccountBook(this, target);
        }
    }
}

```

方法二：破坏不可剥夺条件

上面已经给了你小小的提示，为了解决内置锁的执着，Java 显示锁支持通知(notify/notifyall)和等待(wait)，也就是说该功能可以实现喊一嗓子 **老铁，铁蛋儿的账本先给我用一下，用完还给你** 的功能，

还有，可以通过 加锁时限（线程尝试获取锁的时候加上一定的时限，超过时限则放弃对该锁的请求，并释放自己占有的锁）去解决。

下面是一个类似的例子。

在尝试获取锁的时候加一个超时时间，这也就意味着在尝试获取锁的过程中若超过了这个时限该线程则放弃对该锁请求。若一个线程没有在给定的时限内成功获得所有需要的锁，则会进行回退并释放所有已经获得的锁，然后等待一段随机的时间再重试。这段随机的等待时间让其它线程有机会尝试获取相同的这些锁，并且让该应用在没有获得锁的时候可以继续运行(译者注：加锁超时后可以先继续运行干点其它事情，再回头来重复之前加锁的逻辑)。

以下是一个例子，展示了两个线程以不同的顺序尝试获取相同的两个锁，在发生超时后回退并重试的场景：

```
Thread 1 locks A
Thread 2 locks B
Thread 1 attempts to lock B but is blocked
Thread 2 attempts to lock A but is blocked
Thread 1's lock attempt on B times out
Thread 1 backs up and releases A as well
Thread 1 waits randomly (e.g. 257 millis) before retrying.
Thread 2's lock attempt on A times out
Thread 2 backs up and releases B as well
Thread 2 waits randomly (e.g. 43 millis) before retrying.
```

在上面的例子中，线程2比线程1早200毫秒进行重试加锁，因此它可以先成功地获取到两个锁。这时，线程1尝试获取锁A并且处于等待状态。当线程2结束时，线程1也可以顺利地获得这两个锁（除非线程2或者其它线程在线程1成功获得两个锁之前又获得其中的一些锁）。

需要注意的是，由于存在锁的超时，所以我们不能认为这种场景就一定是出现了死锁。也可能是因为获得了锁的线程（导致其它线程超时）需要很长的时间去完成它的任务。

此外，如果有非常多的线程同一时间去竞争同一批资源，就算有超时和回退机制，还是可能会导致这些线程重复地尝试但却始终得不到锁。如果只有两个线程，并且重试的超时时间设定为0到500毫秒之间，这种现象可能不会发生，但是如果是10个或20个线程情况就不同了。因为这些线程等待相等的重试时间的概率就高的多（或者非常接近以至于会出现问题）。

(译者注：超时和重试机制是为了避免在同一时间出现的竞争，但是当线程很多时，其中两个或多个线程的超时时间一样或者接近的可能性就会很大，因此就算出现竞争而导致超时后，由于超时时间一样，它们又会同时开始重试，导致新一轮的竞争，带来了新的问题。)

这种机制存在一个问题，在Java中不能对synchronized同步块设置超时时间。需要使用JUC的显式锁。

方法三：破坏环路等待条件

破坏环路等待条件,就是按照相同的顺序获得锁。

如果能确保所有的线程都是按照相同的顺序获得锁，那么死锁就不会发生。看下面这个例子：

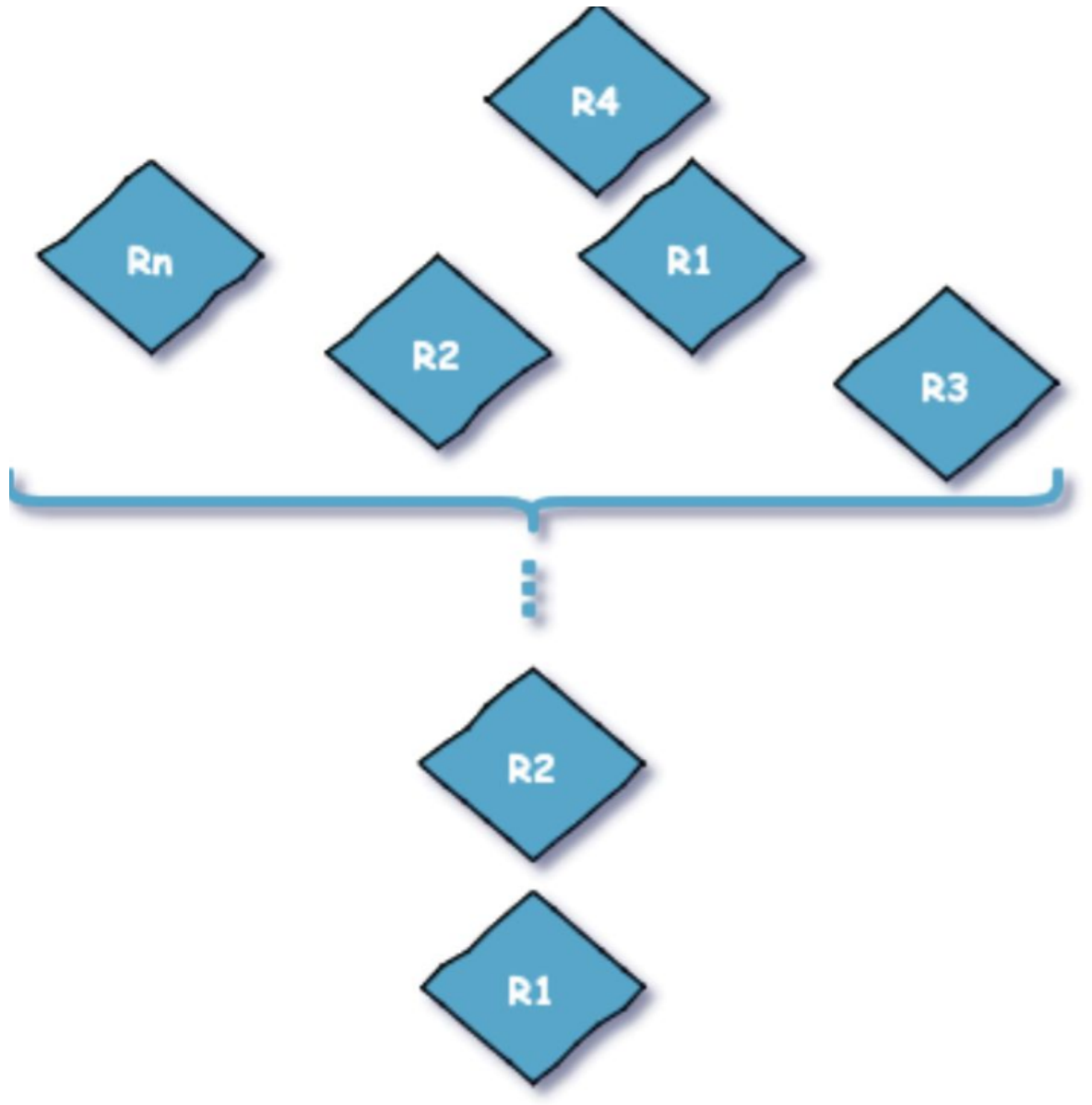
```
Thread 1:
lock A
lock B
Thread 2:
wait for A
lock C (when A locked)
Thread 3:
wait for A
wait for B
wait for C
```

如果一个线程（比如线程3）需要一些锁，那么它必须按照确定的顺序获取锁。它只有获得了从顺序上排在前面的锁之后，才能获取后面的锁。

例如，线程2和线程3只有在获取了锁A之后才能尝试获取锁C(译者注：获取锁A是获取锁C的必要条件)。因为线程1已经拥有了锁A，所以线程2和3需要一直等到锁A被释放。然后在它们尝试对B或C加锁之前，必须成功地对A加了锁。

按照顺序加锁是一种有效的死锁预防机制。但是，这种方式需要你事先知道所有可能会用到的锁(译者注：并对这些锁做适当的排序)，但总有些时候是无法预知的。

破坏环路等待条件也很简单，我们只需要将资源序号大小排序获取就会解决这个问题，将环路拆除



按照id大小的顺序来加锁，先锁住id 小的，然后才锁住 id 大的

```
class Account {
    private int id;
    private int balance;
    // 转账
    void transfer(Account target, int amt){
        Account smaller = this
        Account larger = target;
        // 排序
        if (this.id > target.id) {
            smaller = target;
            larger = this;
        }
        // 锁定序号小的账户
        synchronized(smaller){
            // 锁定序号大的账户
            synchronized(larger){
```

```
        if (this.balance > amt){
            this.balance -= amt;
            target.balance += amt;
        }
    }
}
}
```

当 smaller 被占用时，其他线程就会被阻塞，也就不会存在死锁了。

八 避免死锁

理解了死锁的原因，尤其是产生死锁的四个必要条件，就可以最大可能地避免、预防和解除死锁。所以，在系统设计、进程调度等方面注意如何让这四个必要条件不成立，如何确定资源的合理分配算法，避免进程永久占据系统资源。此外，也要防止进程在处于等待状态的情况下占用资源。因此，对资源的分配要给予合理的规划。

预防死锁和避免死锁的区别：

预防死锁是设法至少破坏产生死锁的四个必要条件之一,严格的防止死锁的出现,而避免死锁则不那么严格的限制产生死锁的必要条件的存在,因为即使死锁的必要条件存在,也不一定发生死锁。避免死锁是在系统运行过程中注意避免死锁的最终发生。

常用避免死锁的方法

有序资源分配法

这种算法资源按某种规则系统中的所有资源统一编号（例如打印机为1、磁带机为2、磁盘为3、等等），申请时必须以上升的次序。系统要求申请进程：

- 1、对它所必须使用的而且属于同一类的所有资源，必须一次申请完；
- 2、在申请不同类资源时，必须按各类设备的编号依次申请。例如：进程PA，使用资源的顺序是R1，R2；进程PB，使用资源的顺序是R2，R1；若采用动态分配有可能形成环路条件，造成死锁。

采用有序资源分配法：R1的编号为1，R2的编号为2；

PA：申请次序应是：R1，R2

PB：申请次序应是：R1，R2

这样就破坏了环路条件，避免了死锁的发生。

银行家算法

详见附录一。

九 检测死锁

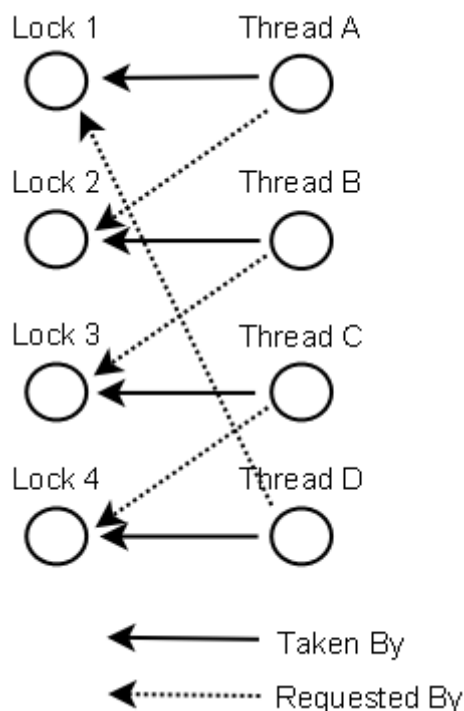
死锁检测是一个更好的死锁预防机制，它主要是针对那些不可能实现按序加锁并且锁超时也不可行的场景。

每当一个线程获得了锁，会在线程和锁相关的数据结构中（map、graph等等）将其记下。除此之外，每当有线程请求锁，也需要记录在这个数据结构中。

当一个线程请求锁失败时，这个线程可以遍历锁的关系图看看是否有死锁发生。例如，线程A请求锁7，但是锁7这个时候被线程B持有，这时线程A就可以检查一下线程B是否已经请求了线程A当前所持有的锁。如果线程B确实有这样的请求，那么就是发生了死锁（线程A拥有锁1，请求锁7；线程B拥有锁7，请求锁1）。

当然，死锁一般要比两个线程互相持有对方的锁这种情况要复杂的多。线程A等待线程B，线程B等待线程C，线程C等待线程D，线程D又在等待线程A。线程A为了检测死锁，它需要递进地检测所有被B请求的锁。从线程B所请求的锁开始，线程A找到了线程C，然后又找到了线程D，发现线程D请求的锁被线程A自己持有。这是它就知道发生了死锁。

下面是一幅关于四个线程（A,B,C和D）之间锁占有和请求的关系图。像这样的数据结构就可以被用来检测死锁。



一般来说，由于操作系统有并发，共享以及随机性等特点，通过预防和避免的手段达到排除死锁的目的是很困难的。这需要较大的系统开销，而且不能充分利用资源。为此，一种简便的方法是系统为进程分配资源时，不采取任何限制性措施，但是提供了检测和解脱死锁的手段：能发现死锁并从死锁状态中恢复出来。因此，在实际的操作系统中往往采用死锁的检测与恢复方法来排除死锁。

死锁检测与恢复是指系统设有专门的机构，当死锁发生时，该机构能够检测到死锁发生的位置和原因，并能通过外力破坏死锁发生的必要条件，从而使得并发进程从死锁状态中恢复出来。

这时进程P1占有资源R1而申请资源R2，进程P2占有资源R2而申请资源R1，按循环等待条件，进程和资源形成了环路，所以系统是死锁状态。进程P1，P2是参与死锁的进程。

下面我们再来看一看死锁检测算法。算法使用的数据结构是如下这些：

占有矩阵A: nm 阶，其中 n 表示并发进程的个数， m 表示系统的各类资源的个数，这个矩阵记录了每一个进程当前占有各个资源类中资源的个数。

申请矩阵R: nm 阶，其中 n 表示并发进程的个数， m 表示系统的各类资源的个数，这个矩阵记录了每一个进程当前要完成工作需要申请的各个资源类中资源的个数。

空闲向量T: 记录当前 m 个资源类中空闲资源的个数。

完成向量F: 布尔型向量值为真 (true) 或假 (false)，记录当前 n 个并发进程能否进行完。为真即能进行完，为假则不能进行完。

临时向量W: 开始时 $W = T$ 。

算法步骤：

(1) $W = T$,

对于所有的 $i=1, 2, \dots, n$,

如果 $A[i]=0$, 则 $F[i] = \text{true}$; 否则, $F[i] = \text{false}$

(2) 找满足下面条件的下标 i :

$F[i] = \text{false}$ 并且 $R[i] \leq W$

如果不存在满足上面的条件 i ，则转到步骤 (4)。

(3) $W = W + A[i]$

$F[i] = \text{true}$

转到步骤 (2)

(4) 如果存在 i ， $F[i] = \text{false}$ ，则系统处于死锁状态，且 P_i 进程参与了死锁。什么时候进行死锁的检测取决于死锁发生的频率。如果死锁发生的频率高，那么死锁检测的频率也要相应提高，这样一方面可以提高系统资源的利用率，一方面可以避免更多的进程卷入死锁。如果进程申请资源不能满足就立刻进行检测，那么每当死锁形成时即能被发现，这和死锁避免的算法相近，只是系统的开销较大。为了减小死锁检测带来的系统开销，一般采取每隔一段时间进行一次死锁检测，或者在CPU的利用率降低到某一数值时，进行死锁的检测。

十 解除死锁

一旦检测出死锁，就应立即采取相应的措施，以解除死锁。

死锁解除的主要方法有：

- 1) 资源剥夺法。挂起某些死锁进程，并抢占它的资源，将这些资源分配给其他的死锁进程。但应防止被挂起的进程长时间得不到资源，而处于资源匮乏的状态。
- 2) 撤销进程法。强制撤销部分、甚至全部死锁进程并剥夺这些进程的资源。撤销的原则可以按进程优先级和撤销进程代价的高低进行。
- 3) 进程回退法。让一（多）个进程回退到足以回避死锁的地步，进程回退时自愿释放资源而不是被剥夺。要求系统保持进程的历史信息，设置还原点。

附录1：银行家算法

背景简介

在银行中，客户申请贷款的数量是有限的，每个客户在第一次申请贷款时要声明完成该项目所需的最大资金量，在满足所有贷款要求时，客户应及时归还。银行家在客户申请的贷款数量不超过自己拥有的最大值时，都应尽量满足客户的需要。在这样的描述中，银行家就好比操作系统，资金就是资源，客户就相当于要申请资源的进程。

银行家算法是一种最有代表性的避免死锁的算法。在避免死锁方法中允许进程动态地申请资源，但系统在进行资源分配之前，应先计算此次分配资源的安全性，若分配不会导致系统进入不安全状态，则分配，否则等待。为实现银行家算法，系统必须设置若干数据结构。

安全序列是指一个进程序列 $\{P_1, \dots, P_n\}$ 是安全的，即对于每一个进程 $P_i (1 \leq i \leq n)$ ，它以后尚需要的资源量不超过系统当前剩余资源量与所有进程 $P_j (j < i)$ 当前占有资源量之和。（即在分配过程中，不会出现某一进程后续需要的资源量比其他所有进程及当前剩余资源量总和还大的情况）

注：存在安全序列则系统是安全的，如果不存在则系统不安全，但不安全状态不一定引起死锁。

原理过程

系统给当前进程分配资源时，先检查是否安全：

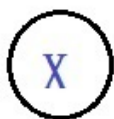
在满足当前的进程 X 资源申请后，是否还能有足够的资源去满足下一个距最大资源需求最近的进程（如某进程最大需要5个单位资源，已拥有1个，还尚需4个），若可以满足，则继续检查下一个距最大资源需求最近的进程，若均能满足所有进程，则表示为安全，可以允许给当前进程 X 分配其所需的资源申请，否则让该进程 X 进入等待。

注：检查过程中，每拟满足一个进程，则进行下个检查时，当前可用资源为回收上一个进程资源的总值，每满足一个进程表示此进程已结束，资源可回收。

如图：

当前剩余资源

Available



当前申请
资源的进程
需要Need个

Pi i=0 1 2 3

5	4	6	3	...	...	...	...	...
---	---	---	---	-----	-----	-----	-----	-----

后续需要资源的进程队列

Available - Need 去检查Pi是否可以满足
若可以，则Available += Pi，
然后去检查下一个Pi 是否可以满足

i为下一个距最大资源最近的进程

若均可满足，则
允许当前进程X
的资源申请，否
则拒绝，让其等
待

算法过程

Available[]矩阵数组表示某类资源的可用量

Claim[i][j]表示进程Pi最大需要Rj类资源的数量

Allocation[i][j]表示Pi已占有的Rj类资源数量

Need[i][j]表示Pi尚需Rj类资源的数量

$Need[i][j] = Claim[i][j] - Allocation[i][j]$

Request[i]表示进程Pi进程的申请向量，如 Request[i][j]=m 表示Pi申请m个Rj类资源

对于当前进程Pi X

(1) 检查if(Request[i][j]<=Need[i][j]) goto (2)

else error("进程 i 对资源的申请量大于其说明的最大值");

(2) 检查 if (Request[i][j]<=Available[j]) goto (3)

else wait(); //注意是等待！即在对后续进程的需求资源判断中，若出现不符合的则安全检查结束，当前进程进入等待

(3) 系统试探地把资源分给Pi 并修改各项属性值（具体是否成立，则根据安全检查的结果）

$Available[j] = Available[j] - Request[i][j]$

$Allocation[i][j] = Allocation[i][j] + Request[i][j]$

$Need[i][j] = Need[i][j] - Request[i][j]$

(4) 安全检查，若检查结果为安全，则（3）中执行有效，否则分配作废，使该Pi进程进入等待

检查算法描述：

向量Free[j]表示系统可分配给各进程的Rj类资源数目，初始与当前Available等值

向量Finish[i]表示进程Pi在此次检查中是否被满足，初始均为false 当有足有资源可分配给进程时，

Finish[i]=true, Pi完成并释放资源(Free[j]+=Allocation[i][j])

1) 从进程队列中找一个能满足下述条件的进程Pi

①、Finish[i]==false,表示资源未分配给Pi进程

②、Need[i][j] < Free[j],表示资源足够分配给Pi进程

2) 当Pi获得资源后, 认为Pi完成, 释放资源

Free[j]+=Allocation[i][j];

Finish[i]=true;

goto Step 1) ;

例:

```
int trueSum=0, i=0 ; boolean Flag=true;

while( trueSum < P.length-1 && Flag == true ) {

    i=i%P.length;

    if( Finish[ i ] == false ){

        if(Need[ i ][ j ] < Free[ j ]){

            Free[ j ]+=Allocation[ i ][ j ];

            Finish[ i ]=true;

            trueSum++;

            i++;

        } else {

            Flag=false;

        }

    }

}
```

if(Flag==false)

检查不通过, 拒绝当前进程X的资源申请

else

检查通过, 允许为当前进程X分配资源

即若可达到Finish[0,1,2,...n] == true 成立则表示系统处于安全状态

例:

有5个进程{P1,P2,P3,P4,P5}。4类资源{R1,R2,R3,R4} 各自数量为6、3、4、2

T0时刻各进程分配资源情况如下

进程\资源	Allocation				Claim				Need				Available			
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4
P1	3	0	1	1	4	1	1	1	1	1	0	0				
P2	0	1	0	0	0	2	1	2	0	1	1	2				
P3	1	1	1	0	4	2	1	0	3	1	0	0	1	0	2	0
P4	1	1	0	1	1	1	2	1	0	0	2	0				
P5	0	0	0	0	2	1	1	0	2	1	1	0				

T0时刻为安全状态，存在安全序列{P4, P1, P2, P3, P5}如下：

进程\资源	Need				Free				Allocation				Free+Allocation				Finish
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	
P4	0	0	2	0	1	0	2	0	1	1	0	1	2	1	2	1	True
P1	1	1	0	0	2	1	2	1	3	0	1	1	5	1	3	2	True
P2	0	1	1	2	5	1	3	2	0	1	0	0	5	2	3	2	True
P3	3	1	0	0	5	2	3	2	1	1	1	0	6	3	4	2	True
P5	2	1	1	0	6	3	4	2	0	0	0	0	6	3	4	2	True

注意：银行家算法在避免死锁角度上非常有效，但是需要在进程运行前就知道其所需资源的最大值，且进程数也通常不是固定的，因此使用有限，但从思想上可以提供了解，可以转换地应用在其他地方。

参考文档：

<https://cloud.tencent.com/developer/article/1541513>

<https://blog.csdn.net/fdoubleman/article/details/97238420>

<http://www.voidcn.com/article/p-gjtwwpmp-ws.html>

<https://blog.csdn.net/hd12370/article/details/82814348>

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：

<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班

定价19999 / 早鸟 3999
即将涨价 4999

已经发布

- 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，2023春招面试涨薪神器
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）——架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、较杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》

规划中

- 《彻底穿透：skywalking 源码（代表链路跟踪）+ Java agent + bytebuddy 探针》
- 《架构师实操篇：基于netty 手写 rpc 框架-参考 dubbo、seata rpc框架》
- 《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》
- 《架构师实操篇：千万级任务调度平台 架构与实操-基于尼恩17年的亿级搜索项目》
- 《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操-基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少坑，少弯路

简历指导

这个很重要，
对于涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

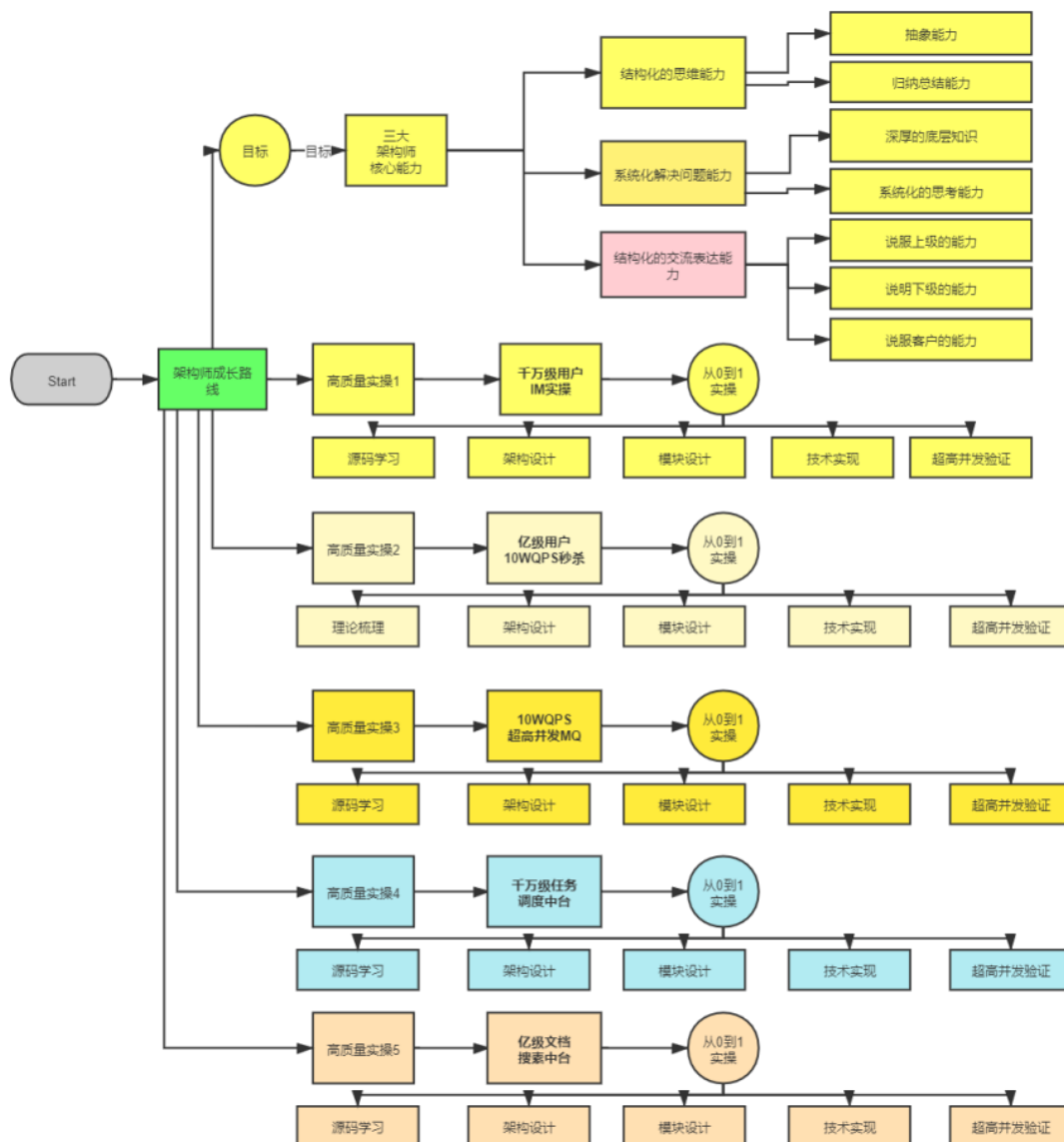
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



【样章】第 17 章:横扫全网Rocketmq 视频第 2 部曲: 工业级 rocketmq 高可用(HA) 底层原理和实操

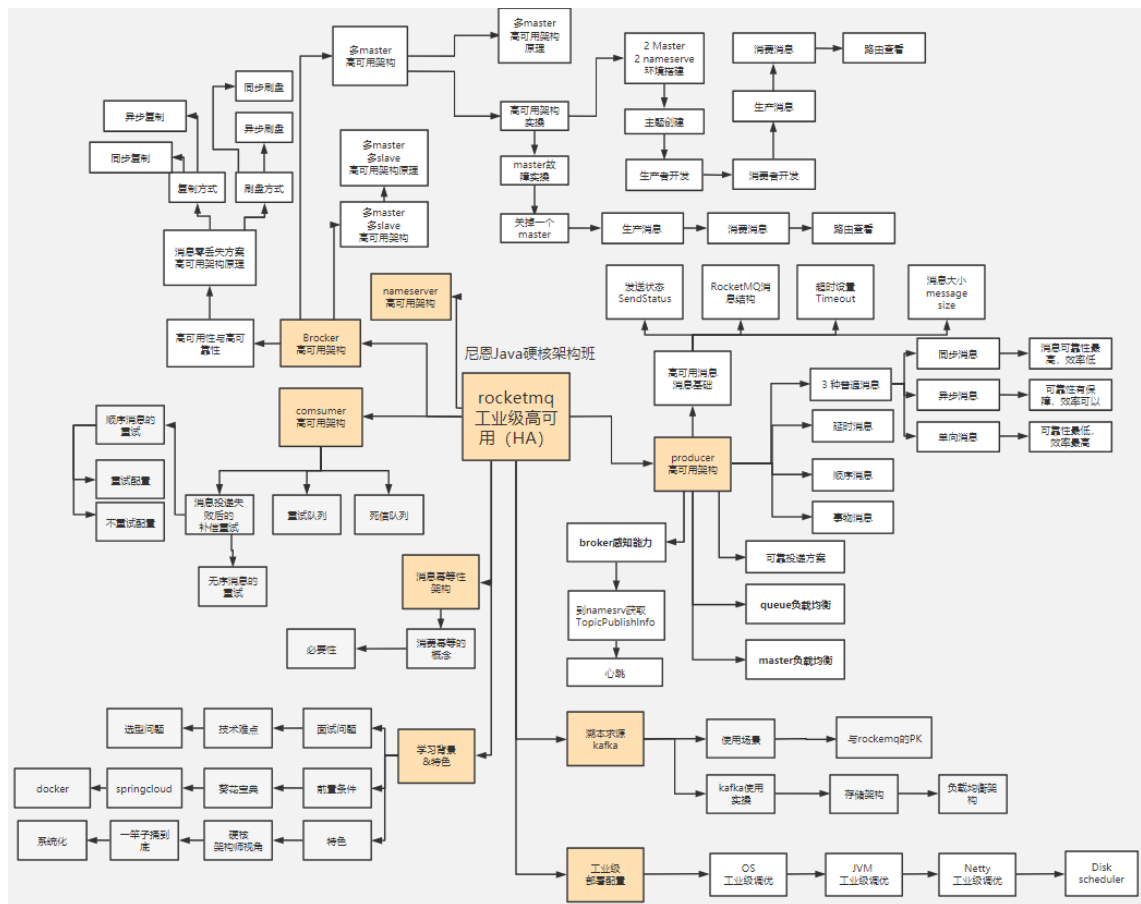
工业级 rocketmq 高可用底层原理, 包含: 消息消费、同步消息、异步消息、单向消息等不同消息的底层原理和源码实现; 消息队列非常底层的主从复制、高可用、同步刷盘、异步刷盘等底层原理。

工业级 rocketmq 高可用底层原理和搭建实操, 包含: 高可用集群的搭建。

解决以下难题:

- 1、技术难题: RocketMQ 如何最大限度的保证消息不丢失的呢? RocketMQ 消息如何做到高可靠投递?
- 2、技术难题: 基于消息的分布式事务, 核心原理不理解
- 3、选型难题: kafka or rocketmq , 该娶谁?

下图链接: <https://www.proceson.com/view/6178e8ae0e3e7416bde9da19>



成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新	最后发表	热门	精华	最新	最后发表	热门	精华
 成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%	 卷王1号	超级版主	前天 17:41	 成功案例：[693号卷王] 二线城市6年卷王喜提4大优质Offer，含央企offer，最高薪酬35W	 卷王1号	超级版主	2022-4-16
 成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W	 卷王1号	超级版主	2022-9-21	 成功案例：[85号卷王] 双非2本小伙，春招大捷，喜提9个offer，最高薪酬近30万	 卷王1号	超级版主	2022-4-14
 成功案例：[878号卷王] 小伙8年经验，年薪60W	 卷王1号	超级版主	2022-8-13	 成功案例：[741号卷王] 卷王逆袭！6年小伙从很少面试机会到搞定35K*14薪Offer	 卷王1号	超级版主	2022-4-12
 年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W	 卷王1号	超级版主	2022-2-11	 成功案例：[642号卷王] 热烈祝贺，6年卷王喜提优质国企offer	 卷王1号	超级版主	2022-4-7
 成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬逆势涨30%	 卷王1号	超级版主	前天 17:43	 成功案例：[796号卷王] 热烈祝贺，36岁卷王喜提52万优质offer	 卷王1号	超级版主	2022-3-25
 成功案例：[250号卷王] 就业极寒时代，收offer 涨25%	 卷王1号	超级版主	前天 17:38	 成功案例：[15号卷王] 小伙卷1年，涨薪9K+，喜收ebay等多个优质offer	 卷王1号	超级版主	2022-3-24
 成功案例：[612号卷王] 就业极寒时代，从外包到白研	 卷王1号	超级版主	前天 17:15	 成功案例：[821号卷王] 小伙狠卷3个月，喜提10多个offer	 卷王1号	超级版主	2022-3-21
 成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W	 卷王1号	超级版主	2022-9-21	 成功案例：[736号卷王] 3年半经验收22k offer，但是小伙志存高远，冲击25k+	 卷王1号	超级版主	2022-3-20
 成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%	 卷王1号	超级版主	2022-9-21	 成功案例：热烈祝贺一群小卷王offer拿到手软，甚至拒了阿里offer	 卷王1号	超级版主	2022-3-16
 成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K	 卷王1号	超级版主	2022-9-21	 简历案例：简历一改，腾讯的邀请就来！热烈祝贺，小伙收到一大堆面试邀请	 卷王1号	超级版主	2022-3-10
 成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%	 卷王1号	超级版主	2022-9-21	 成功案例：祝贺我圈两大超级卷王，一个过了阿里HR面，一个过了阿里2面	 卷王1号	超级版主	2022-3-10
 成功案例：[287号卷王] 不惧大寒流，卷王逆市收4 offer，涨30%，可喜可贺	 卷王1号	超级版主	2022-5-30	 成功案例：小伙伴php转Java，卷1.5年Java，涨薪50%，喜收多个优质offer	 卷王1号	超级版主	2022-3-10
 成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+	 卷王1号	超级版主	2022-7-5	 成功案例：4年小伙狠卷半年，拿到 移动、京东 两大顶级offer	 卷王1号	超级版主	2022-3-5
 成功案例：[7号卷王] 热烈祝贺小伙伴涨薪120%	 卷王1号	超级版主	2022-8-13	 成功案例：[267号卷王] 助力3年经验卷王，拿到蜂巢的17k x 14薪的offer	 卷王1号	超级版主	2022-2-27
 成功案例：[134号卷王] 大三小伙卷1年，斩获顶级央企Offer，成功逆袭	 卷王1号	超级版主	2022-7-6	 成功案例：[143号卷王] 二本院校00后卷神，毕业没到一年跳到字节，年薪45W	 卷王1号	超级版主	2022-2-27
 成功案例：[1008号卷王] 5年经验卷王收42W offer，月涨8000，可喜可贺	 卷王1号	超级版主	2022-5-30	 成功案例：[494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer	 卷王1号	超级版主	2022-2-27
 成功案例：[453号卷王] 非全日制 6年卷王喜提3 offer，年薪30W，可喜可贺	 卷王1号	超级版主	2022-5-21	 成功案例：[76号卷王] 2线城市卷王，狠卷1.5年，喜收22K offer	 卷王1号	超级版主	2022-2-27
 成功案例：[924号卷王] 6年卷王喜提4 offer，最高涨薪9000，可喜可贺	 卷王1号	超级版主	2022-5-21	 成功案例：[429号卷王] 小伙伴在社群卷5个月，涨8k+	 卷王1号	超级版主	2022-2-27
 成功案例：[15号卷王] 4年卷王入职 微软，涨薪50%，可喜可贺	 卷王1号	超级版主	2022-5-12	 成功案例：[154号卷王] 双非学校毕业卷王，连拿 京东到家&滴滴 两个大厂Offer	 卷王1号	超级版主	2022-2-27
 成功案例：[527号卷王] 4年卷王喜提2 offer，涨薪50%，可喜可贺	 卷王1号	超级版主	2022-5-13	 成功案例：[232号卷王] 涨薪10K，继续卷向食物链顶端	 卷王1号	超级版主	2022-2-27
 成功案例：[788号卷王] 3年卷王喜提优质Offer，涨薪60%	 卷王1号	超级版主	2022-5-11	 成功案例：狠卷1年技术，喜收 腾讯、阿里、微软 三大Offer，最高年薪56W	 卷王1号	超级版主	2022-2-27
 成功案例：热烈祝贺：非全日制卷王，喜提2个心仪offer，面3家过2家	 卷王1号	超级版主	2022-4-21	 成功案例：[449号卷王] 应届毕业卷王喜收 滴滴offer，年薪33W	 卷王1号	超级版主	2022-2-27
 成功案例：[732号卷王] 尼恩助力3年经验卷王收获 京东offer，年薪35W	 卷王1号	超级版主	2022-2-27	 成功案例：[551号卷王] 小伙伴学完后，成功进入大厂，并且推荐自己的朋友加VIP学习	 卷王1号	超级版主	2022-2-10
 成功案例：[558号卷王] 2年经验卷王，喜收 网易和阿里子公司两个优质offer	 卷王1号	超级版主	2022-2-27	 成功案例：[214号卷王] 助力2年经验卷王，成功拿到17K月薪	 卷王1号	超级版主	2022-2-10
 成功案例：[569号卷王] 双非应届卷王，喜收字节跳动实习offer	 卷王1号	超级版主	2022-2-25	 成功案例：[92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer	 卷王1号	超级版主	2022-2-10
 成功案例：[420号卷王] 狠卷1年，卷王涨薪80%，涨薪12000元！	 卷王1号	超级版主	2022-2-25	 成功案例：社群卷王小伙伴成功过了滴滴三面 获滴滴Offer	 卷王1号	超级版主	2022-2-10
 成功案例：[76号卷王] 通过尼恩1年半的指导，专科学历小伙伴从0.8K涨到22K	 卷王1号	超级版主	2022-2-10	 [612号卷王]滴滴小伙伴，蹲点考察半年，觉得靠谱后加入 疯狂创客圈	 卷王1号	超级版主	2022-2-10

简历优化后的成功涨薪案例 (VIP 含免费简历优化)

简历优化，卷王逆袭部分成功案例

The grid displays 20 individual success stories, each with a title, a timeline of resume updates and offers received, and a summary of the outcome. The cases are as follows:

- 小伙8年经验 年薪60W**: 7月12日改简历, 8月10日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 30%.
- 7年经验卷王 薪酬涨30%**: 7月11日改简历, 9月1日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 30%.
- 4年经验卷王逆袭 被毕业后, 反涨24W**: 7月改简历, 8月30日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 24W.
- 小伙5月份“被毕业”, 改简历后 新获顶级央企Offer 涨薪7000+**: 5月29日改简历, 7月5日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 7000+.
- 5年卷王喜收2大Offer 最高涨5K**: 5月19日改简历, 9月13日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 5K.
- 6年小伙伴 年薪40W**: 9月6日改简历, 9月21日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 40W.
- 卷王逆袭成功案例 6年小伙从很少面试机会到 搞定35K*14薪**: 3月9日改简历, 4月11日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 35K*14薪.
- 卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W**: 2月9日改简历, 4月15日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 35W.
- 卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k, 年薪35W**: 4月14日改简历, 5月17日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 9k, 35W.
- 卷王逆袭成功案例 5年经验小伙收2个offer 最高涨薪8k, 年薪42W**: 5月9日改简历, 5月30日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 8k, 42W.
- 小伙高中学历 薪酬涨120%**: 5月6日改简历, 7月22日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 120%.
- 卷王逆袭成功案例 寒冬冻六之际卷王大逆袭 收3大offer, 涨30%**: 5月17日改简历, 5月27日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 30%.
- 卷王逆袭成功案例 4年卷王入职微软, 涨50%**: 3月7日改简历, 5月12日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 50%.
- 4年小伙喜收百度、Boss直聘 等N个顶级Offer 最高涨幅100%**: 6月27日改简历, 9月19日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 100%.
- 卷王逆袭成功案例 4年卷王入收2个offer, 涨50%**: 3月23日改简历, 5月12日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 50%.
- 卷王逆袭成功案例 非全日制卷王 面试3家 收2个offer 涨薪30%**: 4月13日改简历, 4月21日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 30%.
- 卷王逆袭成功案例 非全日制 6年经验卷王 喜提3个Offer, 年包30W**: 5月9日改简历, 5月18日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 30W.
- 卷王逆袭成功案例 双非二本小伙伴喜提9大offer**: 2月22日改简历, 4月13日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 9大offer.
- 小伙大三暑期很焦虑 跟着尼恩卷一年 校招新获顶级央企Offer**: 去年5月19日加入VIP, 今年7月5日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 顶级央企Offer.
- 卷王逆袭成功案例 3年经验卷王, 涨60%**: 4月16日改简历, 5月11日接offer. 秘诀: 改简历+群聊卷. 薪资涨幅: 60%.

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>